PHASE 1 – SYNOPSIS & ERD

FOODRESQ

A Microservices-Based Intelligent Food Redistribution and Matching Platform

Project Synopsis

Microservices: Java / Spring Boot

DevOps: Jenkins / Docker

Version Control: Git / GitHub

Submitted By

Aryaa Upadhyay (A010)

Khushi Raval (A024)

Mansi Gupta (A030)

Table of Contents

1. Introduction

2. Problem Statement

3. Proposed Solution

4. Microservices Architecture

5. Research Gap

6. Objectives

7. Scope of the Project

8. Technologies Used

9. Expected Outcome

10. Future Enhancements

11. Technology Summary

12. System Architecture and ERD

13. Database Normalization

14. Proposed Development Workflow

15. Conclusion

1. Introduction

Food wastage is a significant social and logistical problem. Restaurants, college canteens, hostels, hotels, catering services, and event organizers may have surplus edible food after meals or events. At the same time, NGOs, shelters, community organizations, and other verified recipients may require food resources.

The challenge is not only identifying surplus food or finding organizations willing to receive it. Since food is perishable, effective redistribution requires timely coordination between donors and recipients. Factors such as food quantity, remaining shelf life, recipient requirements, urgency, distance, and pickup availability can influence whether a donation can be successfully redistributed.

FoodResQ is proposed as a microservices-based platform that connects surplus food donors with verified recipient organizations. The platform will provide separate services for authentication, donors, recipients, food donations, intelligent matching, logistics, and notifications.

The key feature of FoodResQ will be a multi-factor matching mechanism that ranks potential recipients rather than simply selecting the nearest organization. The proposed system will consider factors such as food expiry, distance, quantity compatibility, urgency, and pickup availability.

The backend will be developed using Java and Spring Boot, while the system will use REST APIs for communication between services. Docker will be used for containerization and Jenkins will be used to implement a CI/CD pipeline for automated building, testing, and deployment.

2. Problem Statement

Existing food redistribution processes often rely on manual communication through phone calls, messages, spreadsheets, or basic donation platforms. Such approaches can make it difficult to coordinate perishable food efficiently.

A simple proximity-based approach may also be insufficient. For example, an NGO may be geographically close to a donor but may not require the available quantity, may not be able to collect the food immediately, or may have a lower level of urgency than another recipient.

Therefore, there is a need for a system that can digitally manage surplus food donations, maintain recipient requirements, match donations with suitable recipients using multiple factors, prioritize food that is close to expiry, coordinate acceptance and pickup, notify relevant users, and provide a scalable microservices architecture with automated software delivery.

3. Proposed Solution

FoodResQ will provide a centralized platform where food donors can register surplus food and recipient organizations can specify their food requirements.

The system will follow this workflow:

Donor creates donation → system identifies eligible recipients → matching service calculates scores → potential recipients are ranked → recipient accepts → pickup is scheduled → donation is delivered → status is updated and users are notified.

For example, if a college has 100 surplus meals with two hours remaining before expiry, the system can prioritize recipients that have an urgent requirement, need a quantity close to the available quantity, are within a reasonable distance, and can arrange pickup quickly.

The matching service will store the individual factors contributing to the match score so that the recommendation is explainable rather than being a black-box decision.

4. Microservices Architecture

FoodResQ is designed around seven independently deployable microservices.

4.1 Authentication & User Service

Responsible for user registration, login, authentication, role-based access, user profiles, and JWT-based authorization.

4.2 Donor Service

Manages donor organizations, contact information, verification status, profiles, and donation history. Potential donors include restaurants, college canteens, hostels, catering organizations, and event organizers.

4.3 Recipient Service

Manages recipient organizations, verification, food requirements, required quantities, urgency levels, pickup availability, and receiving capacity.

4.4 Food Donation Service

Manages food donations, categories, quantities, preparation time, expiry time, pickup deadlines, and donation status.

4.5 Smart Matching Service

Acts as the main research component. It calculates compatibility scores between donations and potential recipients using multiple factors such as distance, expiry, quantity compatibility, urgency, and pickup availability.

4.6 Logistics Service

Coordinates pickup scheduling, pickup confirmation, delivery status, and delivery confirmation. Real-time driver tracking is intentionally outside the initial scope.

4.7 Notification Service

Handles notifications for events such as new matches, donation acceptance, pickup scheduling, delivery completion, and approaching expiry.

5. Research Gap

Food donation and redistribution systems have already been studied, so the proposed project does not claim that food donation platforms are completely new. Instead, the research focus is the intelligent allocation of perishable food.

A basic redistribution system may match donors and recipients primarily according to availability or geographical proximity. However, effective food redistribution involves multiple competing factors.

FoodResQ proposes to investigate whether considering food expiry, distance, quantity compatibility, recipient urgency, and pickup availability can improve allocation efficiency.

The project can compare two approaches: (1) nearest eligible recipient matching and (2) the proposed multi-factor FoodResQ matching algorithm.

Evaluation can use metrics such as average matching time, successful donation rate, food-expiry rate, average distance, unfulfilled recipient demand, and allocation fairness.

6. Objectives

1. To design and develop a microservices-based food redistribution platform.
2. To provide separate services for donors, recipients, food donations, matching, logistics, authentication, and notifications.
3. To enable donors to register and manage surplus food donations.
4. To enable verified recipient organizations to specify food requirements.
5. To develop a multi-factor algorithm for intelligent donor-recipient matching.
6. To consider food expiry, quantity, urgency, distance, and pickup availability during matching.
7. To provide an explainable match score for recommendations.
8. To provide donation acceptance, pickup, and delivery tracking.
9. To implement secure authentication and role-based access.
10. To implement event-based notifications for important donation events.
11. To containerize microservices using Docker.
12. To implement a Jenkins-based CI/CD pipeline for automated build, testing, and deployment.
13. To evaluate the proposed matching approach against a basic distance-based approach.

7. Scope of the Project

In Scope:

- • Donor registration and organization profiles.
- • Recipient registration and verification.
- • Food donation creation and management.
- • Food category, quantity, preparation time, expiry time, and pickup deadline management.
- • Recipient food requirements, urgency, and pickup availability.
- • Multi-factor matching and recipient ranking.
- • Match explanations.
- • Donation acceptance.
- • Pickup scheduling and delivery status.
- • Notifications and expiry alerts.
- • Authentication and role-based access.
- • REST APIs and microservices.
- • Docker containerization.
- • Jenkins CI/CD.
- • Experimental comparison of matching strategies.

Out of Scope:

- • Real-time GPS vehicle tracking.

- • Online payments.
- • Full commercial delivery integration.
- • Automated laboratory food-quality testing.
- • Blockchain.
- • Physical IoT sensors.
- • A complex machine-learning model requiring a large training dataset.

These exclusions keep the project feasible within a one-to-two-month implementation period while leaving clear opportunities for future enhancement.

8. Technologies Used

Programming Language: Java
Backend Framework: Spring Boot
Microservices: Spring Cloud
API Gateway: Spring Cloud Gateway
Service Discovery: Eureka
API Communication: REST / OpenFeign
Authentication: Spring Security and JWT
Database: MySQL or PostgreSQL
ORM: Spring Data JPA / Hibernate
Frontend: React
Build Tool: Maven
Testing: JUnit and Mockito
Containerization: Docker
CI/CD: Jenkins
Version Control: Git and GitHub

9. Expected Outcome

The expected outcome is a working FoodResQ prototype consisting of multiple independently deployable Java Spring Boot microservices.

The system will allow food donors to register surplus food, recipient organizations to specify requirements, and the platform to identify and rank potential recipients.

The intelligent matching service will provide a multi-factor score based on factors such as expiry, distance, quantity, urgency, and pickup availability. The experimental evaluation will compare the proposed matching mechanism with a simpler matching strategy to determine whether multi-factor allocation can improve donation utilization and reduce potential food wastage.

The project will also demonstrate modern software engineering and DevOps practices through microservices architecture, RESTful APIs, database separation, Docker containerization, automated testing, and Jenkins-based CI/CD.

10. Future Enhancements

Potential future enhancements include machine-learning-based demand prediction, food-freshness prediction, real-time GPS tracking, route optimization, a mobile application, integration with food-safety verification systems, predictive estimation of surplus food, real-time analytics, and integration with external NGO or logistics platforms.

11. Technology Summary

Component	Technology
Programming Language	Java
Backend	Spring Boot
Microservices	Spring Cloud
API Gateway	Spring Cloud Gateway
Service Discovery	Eureka
API Communication	REST / OpenFeign
Authentication	Spring Security + JWT
Database	MySQL / PostgreSQL
ORM	Spring Data JPA / Hibernate
Frontend	React
Build	Maven
Testing	JUnit / Mockito
Containerization	Docker
CI/CD	Jenkins
Version Control	Git / GitHub

12. System Architecture and ERD

The following diagrams describe the proposed seven-service architecture and normalized data ownership. In implementation, each microservice will own its own database; cross-service identifiers are represented logically rather than as physical foreign-key constraints between databases.

Figure 1. Overall Microservices Architecture

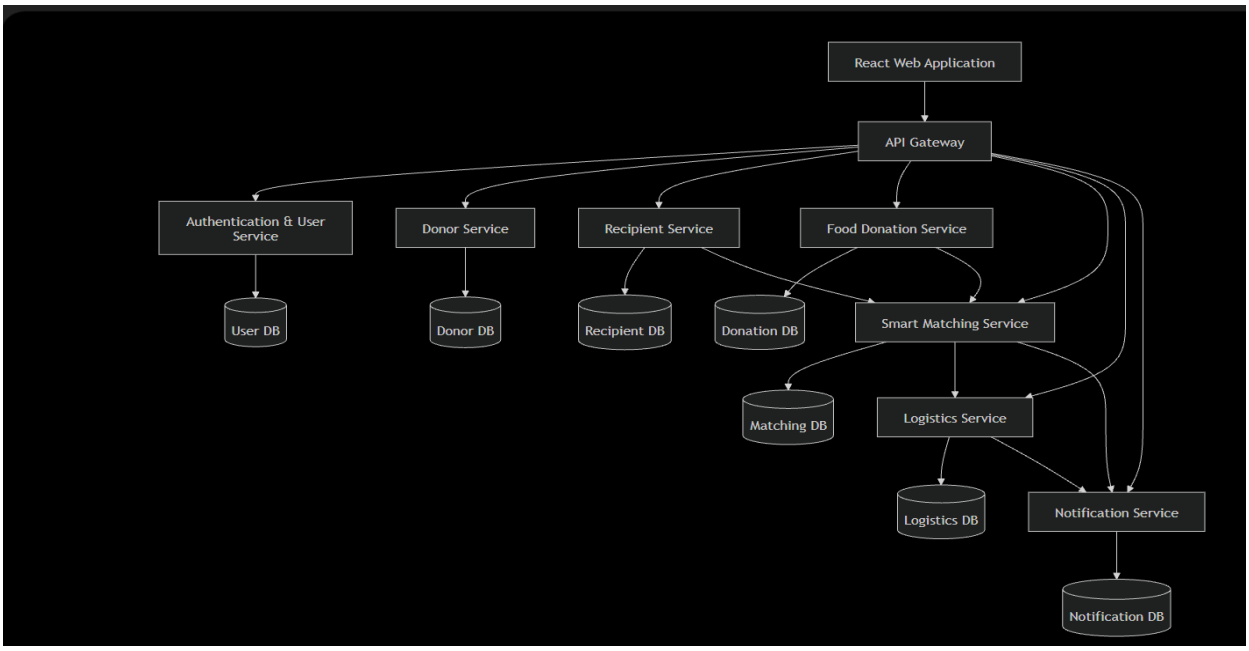


Figure 2. Authentication & User Service ERD

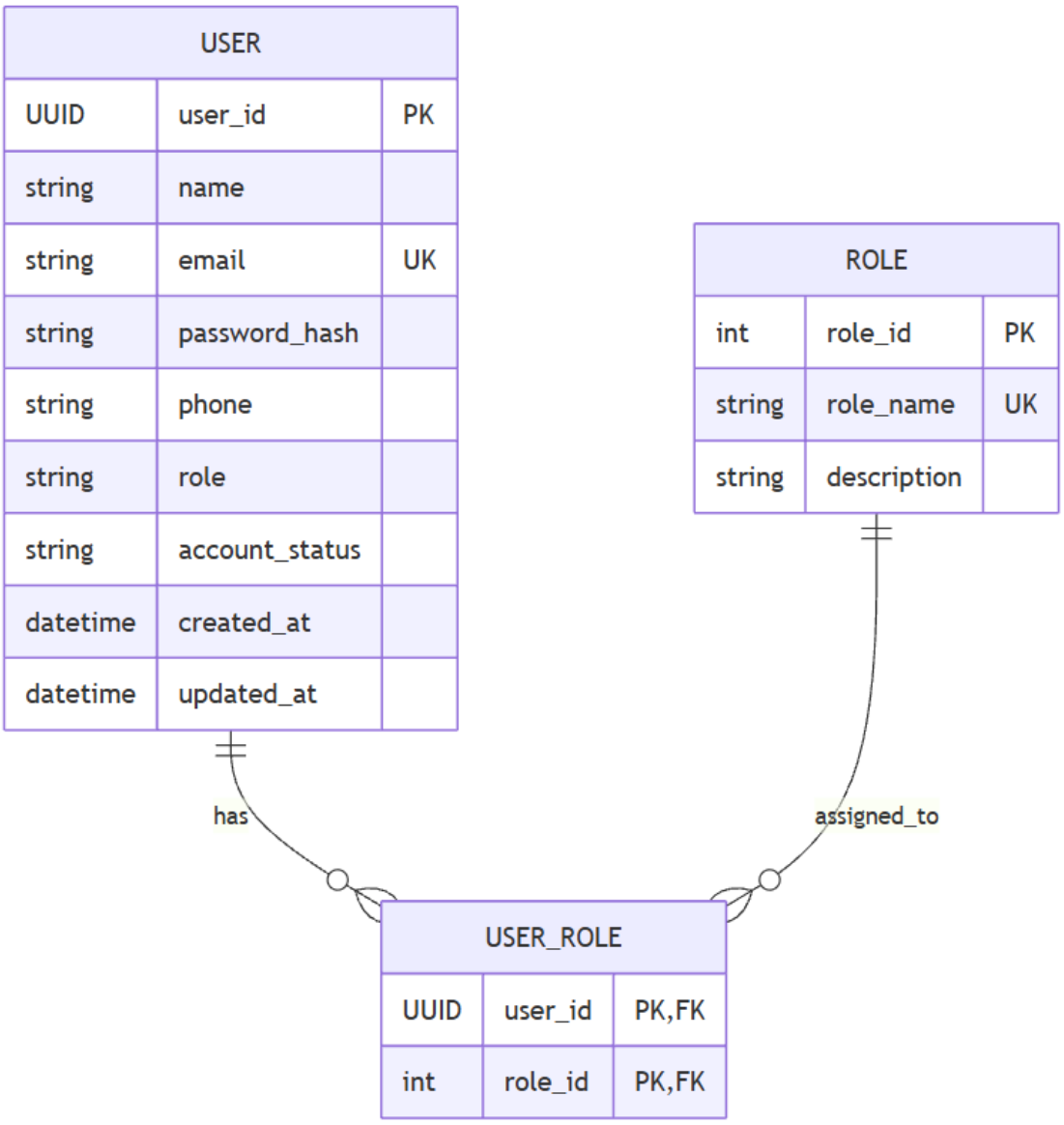


Figure 3. Donor Service ERD

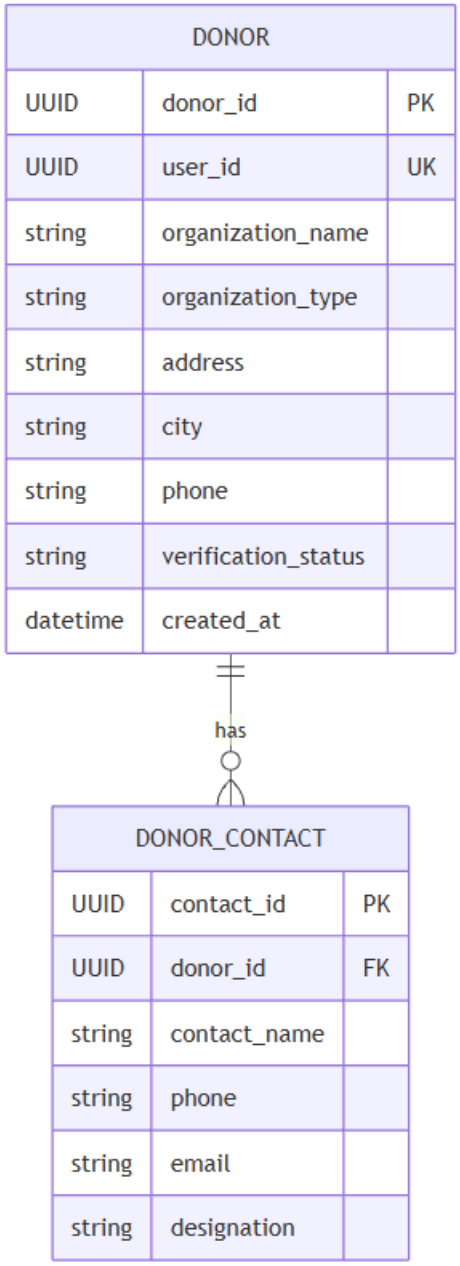


Figure 4. Recipient Service ERD

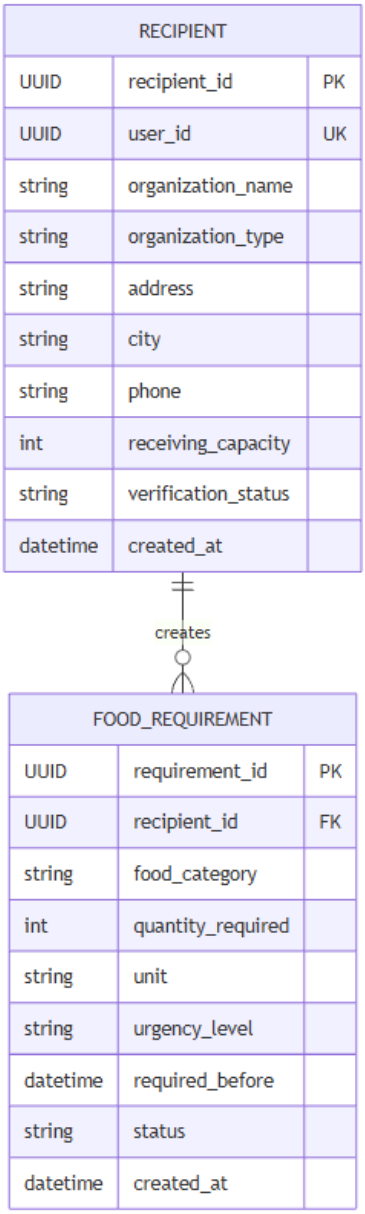


Figure 5. Food Donation Service ERD

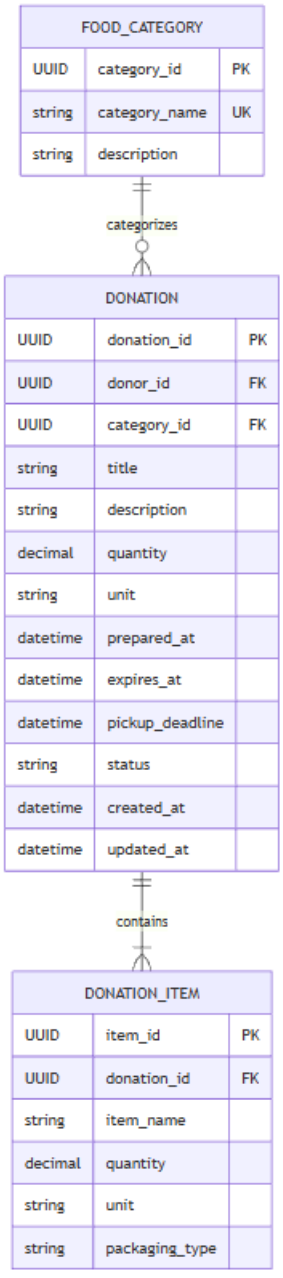


Figure 6. Smart Matching Service ERD

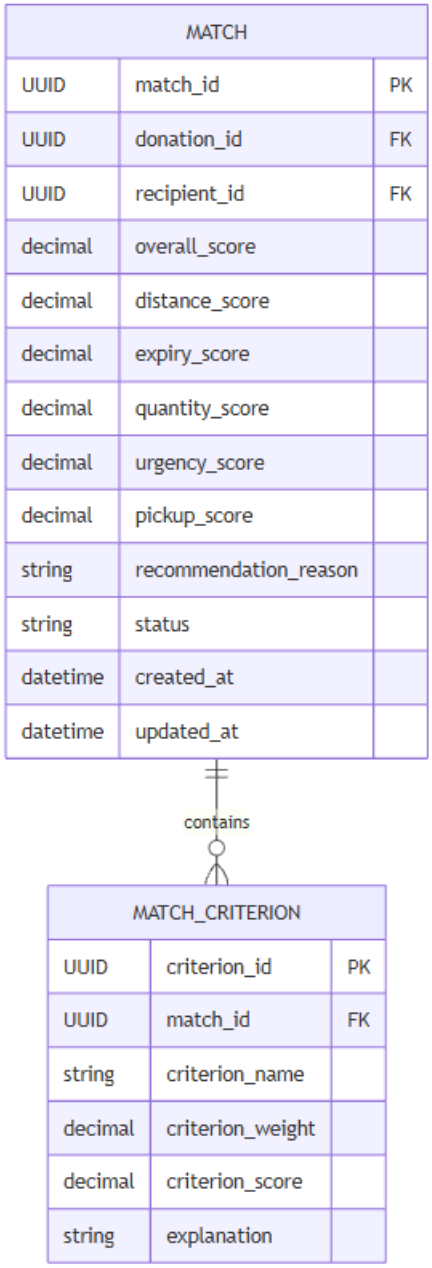


Figure 7. Logistics Service ERD

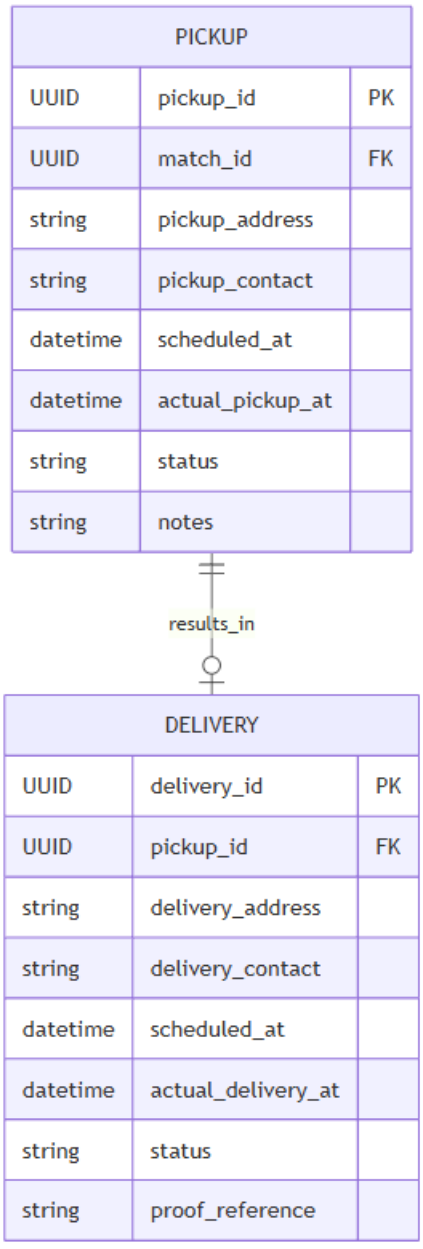


Figure 8. Notification Service ERD

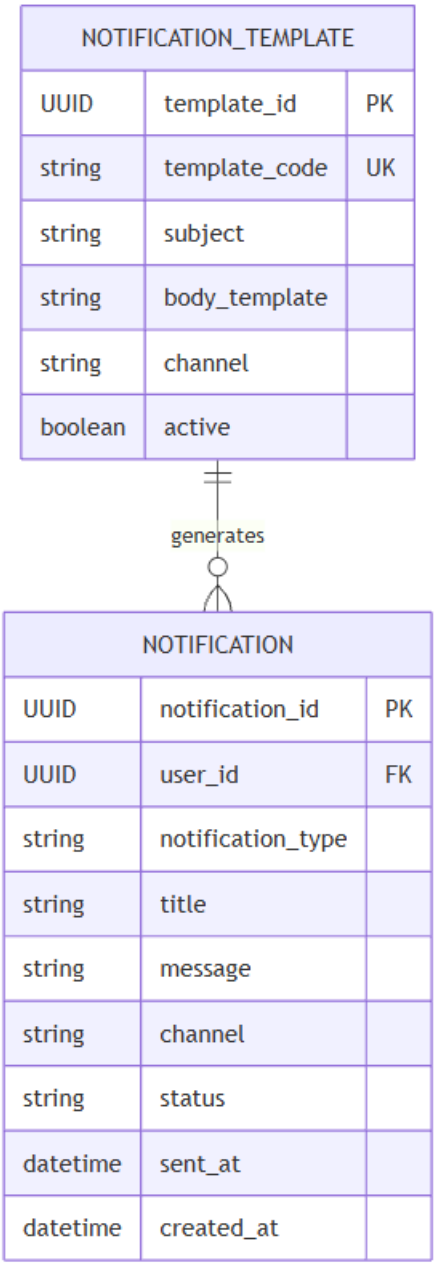
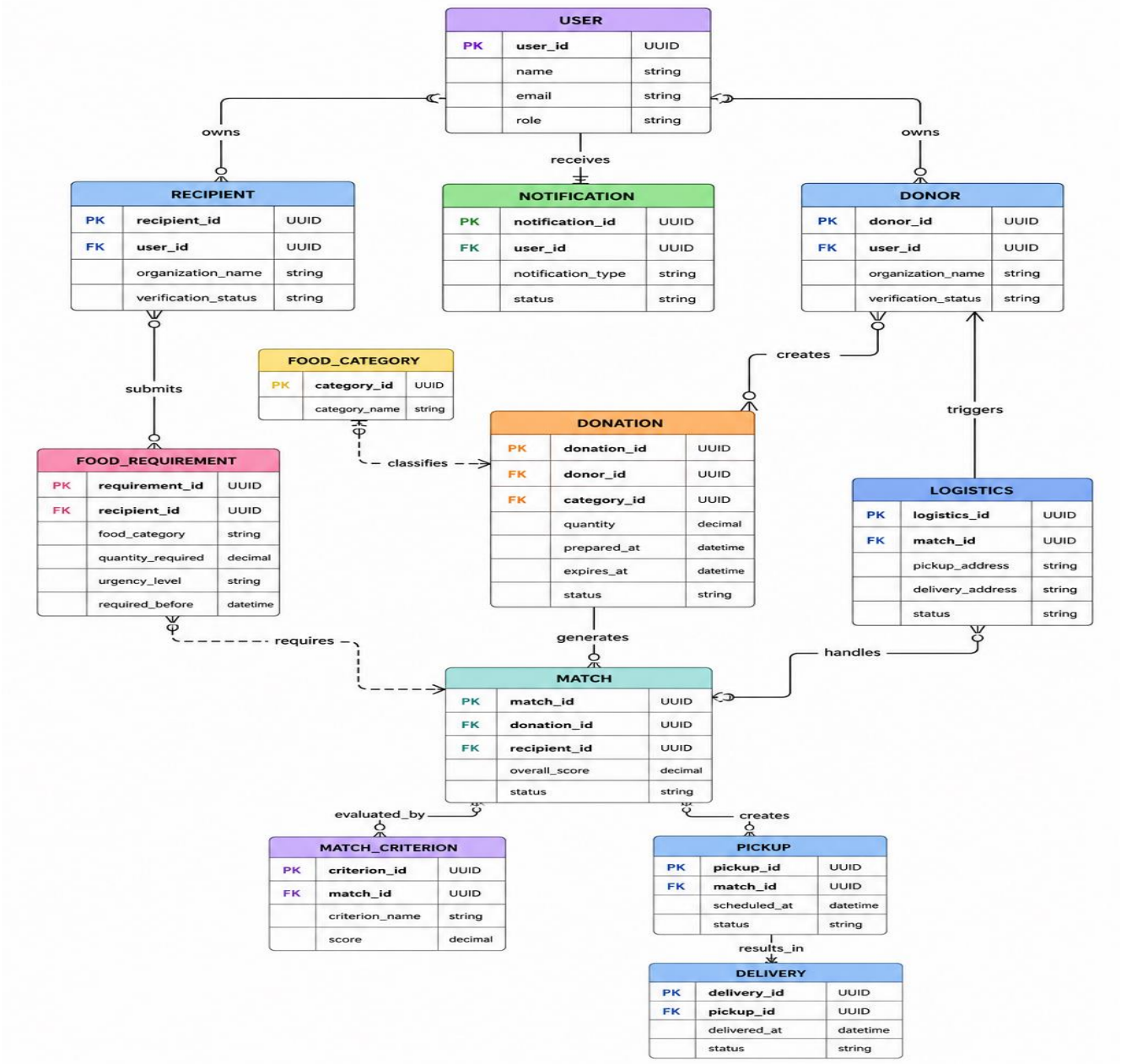


Figure 9. Overall Conceptual ERD



13. Database Normalization

The proposed database design follows normalization principles to reduce redundancy and improve data integrity. Repeating groups are separated into related entities, organization details are separated from contacts and requirements, food categories are represented independently, and matching criteria are stored as individual records. Each microservice owns the data required for its business responsibility. Cross-service references use identifiers such as donor_id, recipient_id, donation_id, and match_id without creating physical foreign-key dependencies across service databases.

14. Proposed Development Workflow

14. Develop each Java Spring Boot microservice independently.

15. Expose REST APIs and integrate services through the API Gateway.
16. Implement authentication and role-based authorization.
17. Create and test the multi-factor matching algorithm.
18. Containerize services with Docker.
19. Write unit and integration tests.
20. Configure Jenkins to checkout, build, test, package, containerize, and deploy the application.
21. Run experimental scenarios comparing distance-only and multi-factor matching.

15. Conclusion

FoodResQ combines a practical social problem with a technically appropriate microservices and DevOps architecture. Its main contribution is the investigation of multi-factor allocation of perishable food rather than simple proximity-based matching. The proposed scope is intentionally limited so that a functional prototype, research evaluation, and Jenkins-based deployment pipeline can be completed within the available development period.